HermSec: A User-Friendly Repository Security Scanner with a Comparison of Static, Single-Agent, Multi-Agent, and Hybrid Review Modes

Seth Whenton
University of Passau
Passau, Germany

Abstract

HermSec is a desktop security scanner for local repositories. It is built to help a developer choose a project, inspect what kind of code it contains, prepare the right scanner tools, run the scan, and generate a readable report. The app includes a chat workflow, live progress cards, scanner settings, model provider settings, doctor checks, automations, and HTML/PDF/JSON reports.

This paper first presents HermSec as a product, then compares four review modes. Deep assisted scan runs static scanners and uses a model to explain scanner evidence. Single agent inspection lets one model inspect bounded code snippets without scanners. MoA inspection uses several specialist agents, a false-positive judge, and an aggregator without scanners. Scanner + MoA inspection runs scanners and MoA together, then uses a judge and final aggregator to validate both sets of findings.

The final controlled run completed 24 of 24 mode-and-fixture runs using efficient non-US models through OpenCode Go. MoA Low had the best F1 score at 0.4762. Single Agent had the best precision at 0.6667 but low recall. Scanner + MoA High had the best recall at 0.5833 but also many false positives. The result shows that multi-agent review can reduce noise compared with scanner-heavy modes, but the hybrid mode still needs stronger candidate filtering.

Keywords

static analysis, software security, large language models, multi-agent systems, vulnerability detection, code review

1 Introduction

Developers need security feedback while they write code. Static application security testing tools can help because they scan source code without running the program [3, 7]. These tools are useful, but they also have limits. They need good rules, language support, and careful tuning. If the rules are weak, the tool can miss real bugs. If the rules are too broad, the tool can report too many false alarms [21].

HermSec is a desktop app that scans projects for security issues. It uses built-in rules and external tools such as Semgrep, Gitleaks, Trivy, Bandit, and other scanners [1, 14, 15, 17]. The app also supports model-assisted review. During testing, one clear problem appeared. Scanner-based reports found many issues, but they could become noisy. Model-only review was easier to read, but it missed more issues.

The main product idea is simple. A user should be able to pick a repository and ask HermSec to scan it. HermSec then inspects the project, checks the language and project files, chooses the scanner tools that fit that project, prepares or installs missing managed tools, runs the scan, and writes a report. This is important because a React project, Java project, Python project, and Rust project should not all need the same scanner setup.

This project studies a simple question: can agent review improve HermSec's scanner reports without losing too much coverage?

The paper compares four HermSec modes:

- **Deep assisted scan**: HermSec runs scanners first. The model then explains scanner evidence when possible.
- **Single agent inspection**: One model reads selected code evidence and writes findings. No scanners run in this mode.
- **MoA inspection**: Several specialist agents inspect the code. A judge checks possible false positives. An aggregator writes the final report.
- **Scanner + MoA inspection**: HermSec runs scanners and MoA review, then a judge and final aggregator validate both sets of findings.

This paper does not claim that agents are always better than scanners. The goal is smaller and clearer. It tests how the four modes behave on controlled examples and explains what should be improved next.

Figures 1 and 2 show parts of the HermSec app that were built during the project. The scan workflow asks the user to choose a mode, shows live progress, and then writes a short result summary. The settings screens let the user configure agent modes and model providers.

2 HermSec Product Overview

HermSec is not only a benchmark script. It is a desktop application for normal developers who want a simple way to check a local project. The main screen is a chat interface. The user selects a project folder, chooses a scan mode, and watches the scan progress in the chat. When the scan ends, HermSec writes report files and gives links to the report folder and PDF.

The product has four main surfaces. The home chat handles project selection, scan mode selection, progress, and final report links. Scanner settings show which tools are installed, enabled, missing, and relevant to the current project. Agent and provider settings let the user choose a provider, model, agent roles, judge behavior, and aggregation behavior. Doctor checks and automations help the user verify readiness before a scan or repeat a known workflow later.

HermSec also has a project inspection step. Before running a scan, it checks the repository files, languages, manifests, lockfiles, and configuration files. From this profile, HermSec chooses the

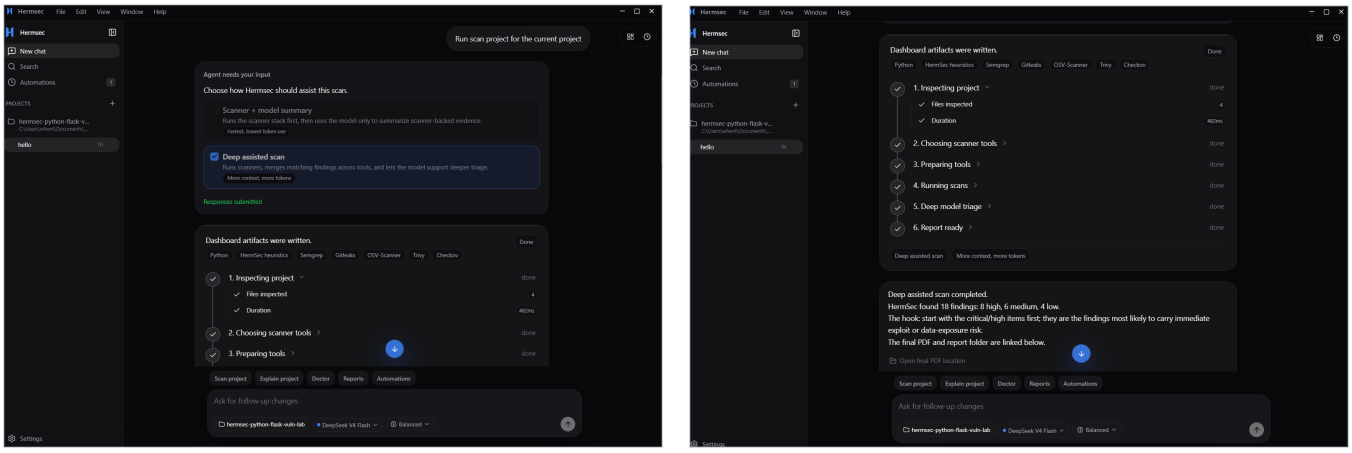

Figure 1: HermSec scan workflow. Left: the user chooses a scan mode and sees live progress. Right: the scan finishes and HermSec shows a short summary with report links.

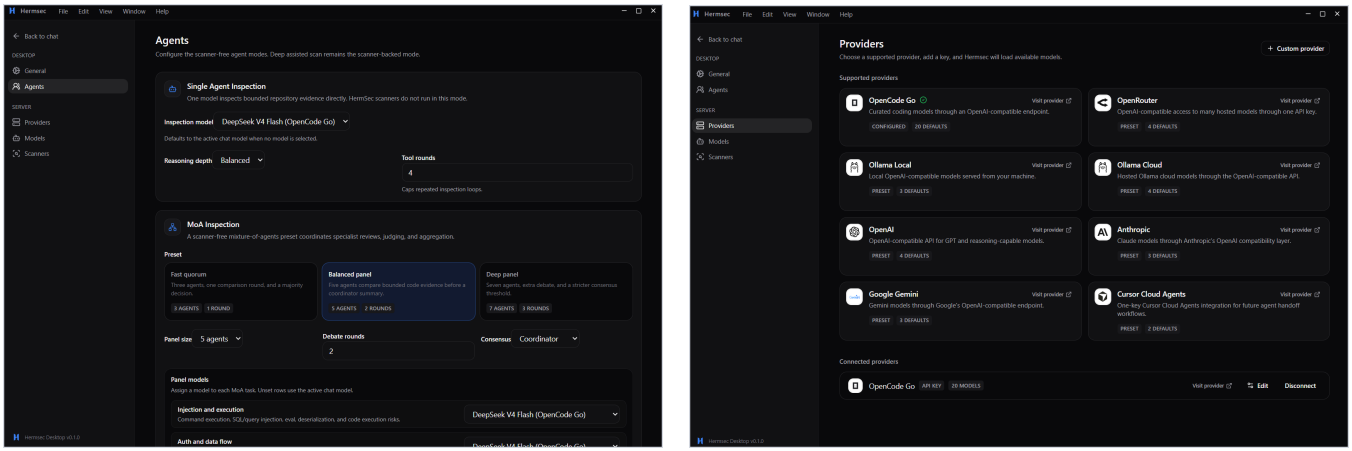

Figure 2: HermSec settings screens. Left: Agents settings for Single Agent and MoA inspection. Right: provider setup with OpenCode Go and other supported providers.

scanners that make sense for the project. For example, a Node.js project can use JavaScript rules, npm audit style checks, secret scanning, OSV, and Trivy. A Python project can use Python rules, Bandit, pip-audit, OSV, and Trivy. This keeps the scan workflow smaller and avoids installing every possible tool for every project.

The scanner manager is part of the product. It shows which scanners are installed, missing, enabled, and used by the current project. Supported scanners can be installed into HermSec's managed tool directory instead of being forced into the user's global system path. Figure 3 shows this scanner settings screen.

Other product features include Doctor checks, model provider setup, model selection, agent settings, scan automations, live progress cards, and HTML/PDF/JSON reports. The Doctor checks confirm that scanner tools, internet connectivity, and provider setup are ready. The report system keeps the final result readable instead of only printing raw scanner output.

3 Background and Related Work

3.1 Static Security Analysis

Static analysis checks code before the program runs. It is often used to find security problems such as injection, unsafe data flow, and dangerous API use [3, 7]. However, static tools must be tested with known examples. A tool may look strong in one project and weak in another project [21].

Security benchmarks help make this testing fair. OWASP Benchmark and NIST Juliet are examples of test suites with known bugs [2, 12]. This project uses the same idea, but with smaller local fixtures. The findings are labelled with CWE identifiers from MITRE CWE [9]. The tested vulnerability types also match common web security risks, such as injection, cross-site scripting, weak cryptography, hardcoded secrets, and unsafe configuration [11].

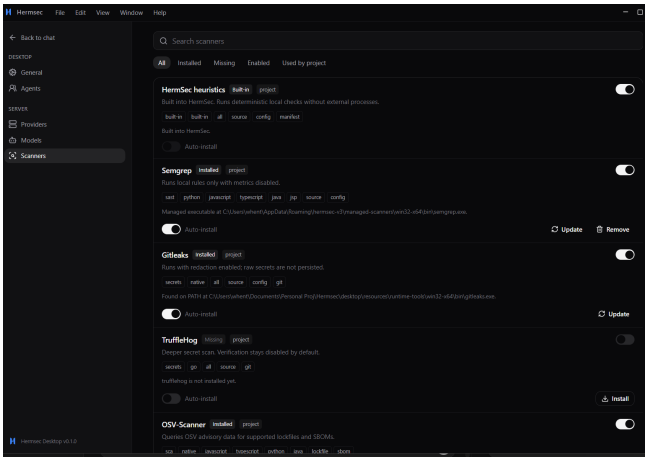

Figure 3: HermSec scanner settings. The app shows installed, missing, enabled, and project-relevant scanners.

3.2 Scanner Harnesses

Many security tools combine several scanners. This is because one scanner usually does not cover every language or every type of vulnerability. HermSec follows this approach. Semgrep gives general static analysis coverage. Gitleaks finds secrets. Trivy finds dependency, secret, and configuration issues. Bandit improves Python coverage [1, 14, 15, 17].

This scanner approach is fast and useful. The main problem is that scanner outputs must be cleaned and merged. If HermSec only lists every scanner result, the user may see duplicates or low-value findings. For this reason, HermSec normalizes findings and stores evidence for each issue.

3.3 LLM Agents and MoA

Large language models can read code and explain risks, but they can also make mistakes [13]. Agent systems try to make model work more controlled by giving the model tools and steps. ReAct is one example of combining reasoning with actions such as searching or reading files [20].

Multi-agent work also shows that several model responses can sometimes improve the final answer [6, 16]. HermSec's MoA mode uses this idea for code security review. Different agents focus on different risk areas. Then a judge removes weak findings, and an aggregator writes the final report. This idea was also inspired by the configurable agent panels in Hermes Agent [10].

4 System Design

4.1 Deep Assisted Scan

Deep assisted scan is the scanner-based mode. HermSec first inspects the project. It chooses scanner tools, prepares them, runs them, and normalizes the findings. After that, the model can explain the scanner-backed evidence. If the model step fails, HermSec can still write a report from scanner findings. This is the closest mode to the original HermSec idea: a simple repository scanner that selects the right security tools for the project.

4.2 Single Agent Inspection

Single agent inspection does not run scanners. One model inspects the repository using safe read-only tools. It can list files, search code, and read snippets. It must report the file path, line or snippet, vulnerability category, evidence, confidence, and remediation. HermSec checks the evidence before adding the finding to the report.

4.3 MoA Inspection

MoA means mixture of agents. In this mode, HermSec runs a panel of specialist agents. The default roles include injection, secrets and supply chain, authentication, crypto and data protection, configuration and IaC, language and framework, API security, and database security. A false-positive judge checks the candidates. The aggregator then writes the final accepted findings.

4.4 Scanner + MoA Inspection

Scanner + MoA inspection is the hybrid mode added after the first tests. In this mode, scanners run and produce candidate findings. MoA agents also inspect the repository with bounded read-only code tools. Then a false-positive judge reviews both scanner findings and agent findings together. Finally, an aggregator writes the accepted and deduplicated findings into the normal HermSec report. The goal is to keep the broad coverage of scanners while using agents to reduce false positives and improve the final explanation.

4.5 Safety Limits

The agent modes are limited on purpose. They cannot run shell commands. They cannot install packages. They cannot execute project scripts. They cannot read outside the selected repository. HermSec also skips noisy folders such as `.git`, `node_modules`, build folders, and vendored code. These limits make the agent modes safer and easier to test.

5 Methodology

5.1 Research Questions

The study asks four questions:

- **RQ1** Which mode gives the best precision, recall, and F1 score?
- **RQ2** Does MoA reduce false positives compared with the scanner-based mode while finding more bugs than one agent?
- **RQ3** Does Scanner + MoA improve the balance by combining scanner coverage with agent filtering?
- **RQ4** What should be improved before testing on larger benchmarks?

5.2 Datasets

Two datasets were used.

The first dataset came from tests prepared by my colleague Ghazal. It covered a broader set of languages and was useful for identifying where the scanner harness needed stronger language-specific coverage. In this refresh, it is treated as exploratory product context only. The paper no longer carries forward its old aggregate scanner-only counts or rates.

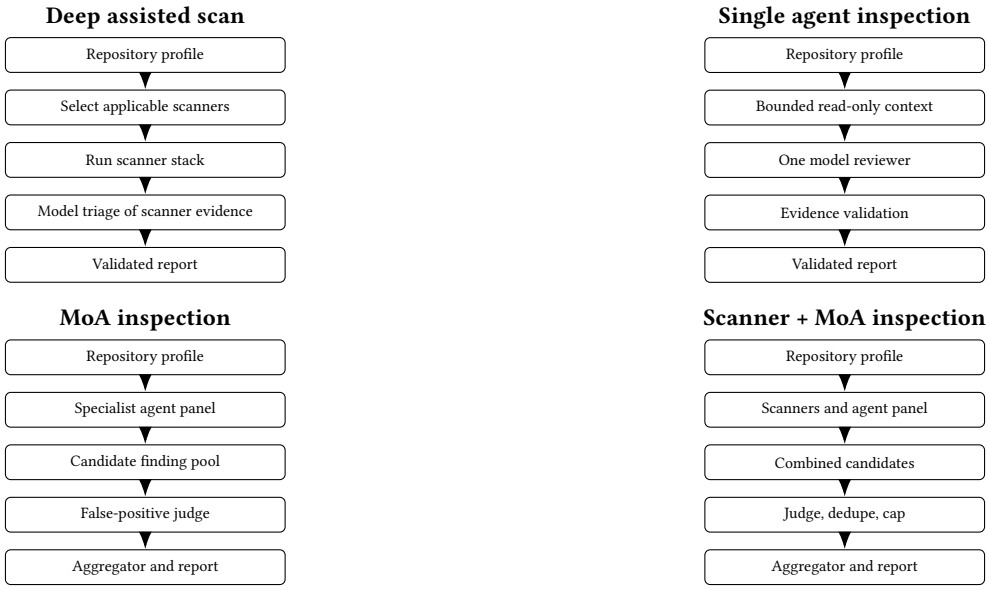

Figure 4: Execution diagrams for the four review modes. The diagrams separate scanner-backed review, one-agent direct review, multi-agent review, and the hybrid path that joins scanner and agent candidates before judging and aggregation.

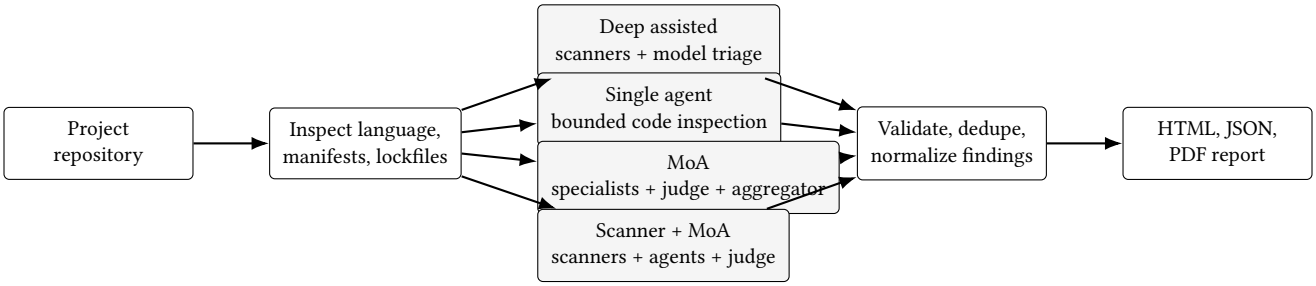

Figure 5: High-level HermSec harness flow. Every scan starts by inspecting the project. Then the selected mode runs and the results are validated, normalized, and written into the normal report template.

The second dataset is the controlled fixture set used for the final mode comparison. The refreshed run used four fixtures with 12 expected findings in total. The fixture categories were:

- vulnerable Node.js/Express project,
- clean Node.js/Express project,
- vulnerable Python/Flask project,
- clean Python/Flask project.

The clean projects had no expected vulnerabilities. The vulnerable projects had six expected findings each. The answer keys and run records are stored in `results/latest/results.json`.

5.3 Harness Steps

The HermSec harness follows the same basic steps for every scan. First, it validates the local project path. Second, it walks the repository and records source files, languages, manifests, lockfiles, and ignored folders. Third, it chooses the scan plan. In scanner-based modes, it selects the scanners that match the project. In agent modes, it prepares bounded file, search, and snippet context. Fourth, it runs the selected mode with timeouts and progress events. Fifth, it normalizes findings into one schema. Finally, it writes the normal HermSec report.

The normal finding schema stores the title, category, severity, confidence, evidence, remediation, tool, rule ID, CWE, location, and fingerprint. This matters because scanner output and agent output must be scored and reported in the same format.

5.4 Execution Setup

The final comparison was run with HermSec V3 on Windows and recorded in `results/latest/results.json`. The artifact includes the configured provider, model names, fixture identifiers, run status, fallback status, finding counts, and scored outcomes for every mode. The latest artifact has `executionMode` set to `actual`, so it is the measurement source used in this paper.

Deep assisted, single agent, MoA, and Scanner + MoA were run across the same controlled fixture set. The Scanner + MoA run used the real CLI mode: scanners ran first, MoA inspection ran next,

combined candidates were judged, and the final report was written through the normal report template.

The Deep assisted provider timeout issue found during earlier smoke tests was addressed by using smaller model explanation chunks and a longer summary watchdog. In the final artifact, two vulnerable Deep assisted rows still used fallback explanations because the model text was rejected as unsupported by the evidence validator. This is safer than accepting invented evidence, but it means the Deep assisted results should be read mainly as scanner-backed findings with guarded model support.

5.5 Model and Cost Choices

Another core idea in HermSec is cost control. Static scanners are the cheapest part because they run locally or through managed command-line tools. Model-assisted modes can become expensive because every agent reads project context and writes structured findings. For that reason, HermSec was designed to use efficient models first and reserve larger models for aggregation only when needed.

The model routing is intentionally open to efficient non-US providers and models exposed through OpenCode Go or similar provider adapters. `deepseek-v4-flash` is useful for this design because DeepSeek documents context caching and pricing behavior that can reduce repeated prompt-prefix cost when the same system prompts and report schemas are reused [4, 5]. Other efficient route models can be recorded in the result artifact's model policy. This matters for HermSec because Single Agent, MoA, and Scanner + MoA repeat similar instructions across fixtures.

`mimo-v2.5` and `minimax-m3` are also relevant to the cost-control direction. MiMo V2.5 is described as an agent-capable long-context model with hybrid attention and KV-cache efficiency [18, 19]. MiniMax presents M3 as a coding and agentic model with long-context support [8]. The final run used `deepseek-v4-flash` for Deep assisted and Single Agent. MoA and Scanner + MoA used `deepseek-v4-flash` and `mimo-v2.5` for specialist agents, `deepseek-v4-pro` for the false-positive judge, and `minimax-m3` for the aggregator.

5.6 Scoring

Each finding was compared with the ground truth by file path and CWE. A true positive means HermSec found a planted issue. A false positive means HermSec reported an issue that did not match the answer key. A false negative means HermSec missed a planted issue.

The metrics were calculated as follows:

$$Precision = \frac{TP}{TP + FP},$$

$$Recall = \frac{TP}{TP + FN},$$

$$F1 = \frac{2 \times Precision \times Recall}{Precision + Recall}.$$

5.7 Problems Faced During Testing

There were a few relevant problems during testing. First, the Deep assisted explanation step initially hit provider timeout failures. This was improved by using smaller chunks and a longer watchdog. In the final run, two Deep assisted vulnerable rows still used fallback explanations because the model output did not pass evidence validation. Second, Scanner + MoA high initially failed when the false-positive judge received too many candidates in one request. This was fixed by batching the judge step and splitting failed batches. Third, scanner-heavy modes produced many findings outside the small answer key. Some of those findings may still be useful in a real project, but benchmark scoring counted only matches defined by the ground truth.

During app development, the main challenge was making the product workflow clear. HermSec needed to show live progress, manage scanners, keep model settings simple, and still write a normal report at the end. The solution was to keep one report format for all modes and add mode metadata such as agents used, judge counts, and source labels.

6 Results

6.1 Refresh Status

The refreshed scoring artifact is available at `results/latest/results.json`. The old broad scanner-only aggregate and the earlier per-language chart are intentionally omitted. They were useful for development, but they are not part of the final comparison.

The final run completed 24 of 24 mode-and-fixture runs. The metrics in this section come from `results.json` and `metrics.csv`. The chart files `mode-metrics.svg` and `mode-counts.svg` were regenerated from the same artifact.

6.2 Aggregate Mode Comparison

Table 1 shows the aggregate comparison. The best F1 score came from MoA Low. Single Agent had the highest precision but missed many planted findings. Scanner-heavy modes found more true positives but also produced many false positives against this small answer key.

6.3 Per-Fixture Results

Table 2 gives a compact per-fixture view. Clean fixtures show whether a mode stayed quiet or produced extra findings.

7 Discussion

7.1 Answer to RQ1

For RQ1, the answer depends on the goal. Single Agent had the best precision at 0.6667, but it only found 2 of 12 expected issues. Scanner + MoA High had the best recall at 0.5833, but it had many false positives. MoA Low had the best F1 score at 0.4762, so it was the best balanced mode in this run.

7.2 Answer to RQ2

For RQ2, MoA improved the balance compared with both Deep assisted and Single Agent. MoA Low found 5 true positives with 4 false positives. Deep assisted found 6 true positives, but it also reported 83 false positives. Single Agent had only 1 false positive, but it missed 10 expected issues. This means the MoA judge and aggregator helped reduce noise, but the system still lost some recall.

Table 1: Aggregate results across the controlled fixture set.

Scenario	Runs	Reported	TP	FP	FN	Precision	Recall	F1
Deep assisted	4	89	6	83	6	0.0674	0.5000	0.1188
Single Agent	4	3	2	1	10	0.6667	0.1667	0.2667
MoA Low	4	9	5	4	7	0.5556	0.4167	0.4762
MoA High	4	6	3	3	9	0.5000	0.2500	0.3333
Scanner + MoA Low	4	74	6	68	6	0.0811	0.5000	0.1395
Scanner + MoA High	4	70	7	63	5	0.1000	0.5833	0.1707

Table 2: Per-fixture results from the refreshed comparison.

Fixture	Mode	Reported	Expected	TP	FP	FN	F1	Notes
Node vuln	Deep	25	6	4	21	2	0.2581	fallback explanation
Node clean	Deep	6	0	0	6	0	0.0000	noisy scanner findings
Python vuln	Deep	57	6	2	55	4	0.0635	fallback explanation
Python clean	Deep	1	0	0	1	0	0.0000	one extra finding
Node vuln	Single	2	6	1	1	5	0.2500	precise but missed issues
Node clean	Single	0	0	0	0	0	1.0000	clean
Python vuln	Single	1	6	1	0	5	0.2857	clean but low recall
Python clean	Single	0	0	0	0	0	1.0000	clean
Node vuln	MoA Low	5	6	2	3	4	0.3636	balanced
Node clean	MoA Low	0	0	0	0	0	1.0000	clean
Python vuln	MoA Low	4	6	3	1	3	0.6000	best vulnerable row
Python clean	MoA Low	0	0	0	0	0	1.0000	clean

7.3 Answer to RQ3

For RQ3, Scanner + MoA did not beat MoA Low on F1 in this controlled run. Scanner + MoA High found the most true positives with 7 of 12, but it also produced 63 false positives. This means the hybrid preserved more scanner coverage, but the judge and aggregator did not reduce scanner noise enough yet.

7.4 Answer to RQ4

For RQ4, the next step is better coverage and more stable evaluation. HermSec still needs stronger language-specific rules and scanners for PHP, Go, Ruby, Rust, C#, C/C++, and configuration files. The agent modes should also be tested on larger public benchmarks. Deep assisted scan should continue reporting fallback reasons clearly. Scanner + MoA should use stricter scanner candidate filtering because too many scanner candidates made the final result noisy.

8 Threats to Validity

This study is small. It used four controlled fixtures and 12 expected findings. The results show a useful direction rather than proving that the same behavior will happen on all projects.

The bugs are synthetic. Real projects can have more complex code and more complex data flow. The scoring method can also count a useful scanner finding as a false positive if it is not in the answer key. Finally, model behavior can change because of provider errors, prompt changes, timeout settings, and model updates.

9 Future Work

Future work should test HermSec on larger benchmarks. Good next benchmarks are OWASP BenchmarkJava, OpenSSF CVE Benchmark, CASTLE, and selected Juliet test cases. HermSec should also improve Java taint tracking, scanner installation, benchmark exports, and cost tracking for model runs. Another useful test would compare different MoA panel sizes, different judge settings, and different Scanner + MoA candidate limits.

10 Conclusion

This paper presents HermSec as a user-friendly repository security scanner and compares four HermSec review modes: Deep assisted scan, Single Agent inspection, MoA inspection, and Scanner + MoA inspection.

The controlled run shows that no mode is perfect. Single Agent was the most precise, but it missed many issues. Scanner-heavy modes found more true positives, but they produced too much noise on the small answer key. MoA Low gave the best F1 score, so it was the best balanced mode in this experiment. Scanner + MoA High had the best recall, but it still needs better filtering before it can beat MoA on overall quality.

The main lesson is that HermSec should keep both scanner-backed and agent-backed workflows. Scanners give broad deterministic coverage. Agents help review evidence and reduce noise. The next version should improve scanner candidate filtering, Java and multi-language coverage, and larger benchmark evaluation.

Acknowledgments

This work was developed for Security Insider Lab II. I thank my colleague Ghazal for preparing broader fixture tests and benchmark notes that helped guide the research refresh.

References

[1] Aqua Security. 2026. Trivy: Scanner for Vulnerabilities, Misconfigurations, Secrets, and Licenses. https://github.com/aquasecurity/trivy. Accessed: 2026-06-27.

[2] Tim Boland and Paul E. Black. 2012. *Juliet 1.1 C/C++ and Java Test Suite.* Technical Report. National Institute of Standards and Technology.

[3] Brian Chess and Gary McGraw. 2004. Static Analysis for Security. *IEEE Security & Privacy* 2, 6 (2004), 76–79.

[4] DeepSeek. 2026. API Pricing. https://api-docs.deepseek.com/quick_start/pricing. Accessed: 2026-06-27.

[5] DeepSeek. 2026. Context Caching. https://api-docs.deepseek.com/guides/kv_cache. Accessed: 2026-06-27.

[6] Yilun Du, Shuang Li, Antonio Torralba, Joshua B. Tenenbaum, and Igor Mordatch. 2023. Improving Factuality and Reasoning in Language Models through Multiagent Debate. https://arxiv.org/abs/2305.14325.

[7] V. Benjamin Livshits and Monica S. Lam. 2005. Finding Security Vulnerabilities in Java Applications with Static Analysis. In *Proceedings of the 14th USENIX Security Symposium*.

[8] MiniMax. 2026. MiniMax-M3. https://www.minimax.io/models/text/m3. Accessed: 2026-06-27.

[9] MITRE. 2026. Common Weakness Enumeration. https://cwe.mitre.org/. Accessed: 2026-06-27.

[10] Nous Research. 2026. Hermes Agent. https://github.com/nousresearch/hermes-agent. Accessed: 2026-06-27.

[11] OWASP Foundation. 2021. OWASP Top 10: The Ten Most Critical Web Application Security Risks. https://owasp.org/Top10/. Accessed: 2026-06-27.

[12] OWASP Foundation. 2026. OWASP Benchmark Project. https://owasp.org/www-project-benchmark/. Accessed: 2026-06-27.

[13] Hammond Pearce, Baleegh Ahmad, Benjamin Tan, Brendan Dolan-Gavitt, and Ramesh Karri. 2022. Asleep at the Keyboard? Assessing the Security of GitHub Copilot's Code Contributions. In *Proceedings of the 2022 IEEE Symposium on Security and Privacy*.

[14] PyCQA. 2026. Bandit: Security Linter from PyCQA. https://bandit.readthedocs.io/. Accessed: 2026-06-27.

[15] Semgrep, Inc. 2026. Semgrep: Lightweight Static Analysis for Many Languages. https://semgrep.dev/. Accessed: 2026-06-27.

[16] Junlin Wang, Jue Wang, Ben Athiwaratkun, Ce Zhang, and James Zou. 2024. Mixture-of-Agents Enhances Large Language Model Capabilities. https://arxiv.org/abs/2406.04692.

[17] Zachary Wilson. 2026. Gitleaks: Detect and Prevent Secrets in Git Repositories. https://github.com/gitleaks/gitleaks. Accessed: 2026-06-27.

[18] Xiaomi. 2026. Model Release: MiMo-V2 Series. https://mimo.mi.com/docs/en-US/updates/model. Accessed: 2026-06-27.

[19] XiaomiMiMo. 2026. Full-Pipeline Inference Optimization for MiMo-V2.5 Series. https://mimo.xiaomi.com/blog/mimo-v2-5-inference. Accessed: 2026-06-27.

[20] Shunyu Yao, Jeffrey Zhao, Dian Yu, Nan Du, Izhak Shafran, Karthik Narasimhan, and Yuan Cao. 2023. ReAct: Synergizing Reasoning and Acting in Language Models. https://arxiv.org/abs/2210.03629.

[21] Misha Zitser, Richard Lippmann, and Tim Leek. 2004. Testing Static Analysis Tools Using Exploitable Buffer Overflows from Open Source Code. In *Proceedings of the 12th ACM SIGSOFT International Symposium on Foundations of Software Engineering*. 97–106.